Part 1. Modeling

We begin this part by presenting the mathematical model for a Transformer in Section 2. Throughout the paper we focus on a simplified version that includes the self-attention mechanism as well as layer normalization (Section 2.2), but excludes additional feed-forward layers commonly used in practice (Section 2.3). This nonetheless leads to a highly nonlinear mean-field interacting particle system. In turn, this system implements, via the continuity equation, a flow map from initial to terminal distributions of particles that we present in Section 3.

2. Interacting particle system

Before writing down the Transformer model, we first provide a brief preliminary discussion to clarify our methodological choice of treating the discrete layer indices in the model as a continuous time variable in Section 2.1, echoing previous work on ResNets. The specifics of the toy Transformer model are presented in Section 2.2, and a complete model is presented in Section 2.3.

2.1. Residual neural networks.

One of the standard paradigms in machine learning is that of supervised learning, where one aims to approximate an unknown function $f : \mathbb{R}^d \to \mathbb{R}^m$, from data, $\mathcal{D} = \{x^{(i)}, f(x^{(i)})\}_{i\in[N]}$ say. This is typically done by choosing one among an arsenal of possible parametric models, whose parameters are then fit to the data by means of minimizing some user-specified cost. With the advent of graphical processing units (GPUs) in the realm of computer vision [KSH12], large neural networks have become computationally accessible, resulting in their popularity as one such parametric model.

Within the class of neural networks, *residual neural networks* (ResNets for short) have become a staple DNN architecture since their introduction in [HZRS16]. In their most basic form, ResNets approximate a function f at $x \in \mathbb{R}^d$ through a sequence of affine transformations, a component-wise nonlinearity, and skip connections. Put in formulae,

$$\begin{cases} x(k+1) = x(k) + w(k)\sigma(a(k)x(k) + b(k)) & \text{for } k \in \{0, \dots, L-1\} \\ x(0) = x\,. \end{cases} \tag{2.1}$$

Here σ is a Lipschitz function applied component-wise to the input vector, while $\theta(\cdot) = (w(\cdot), a(\cdot), b(\cdot)) \in \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^d$ are trainable parameters. We say that (2.1) has $L \geqslant 1$ hidden layers (or $L+1$ layers, or is of depth L). The output $x_i(L)$ serves as a *representation* of the input $x^{(i)}$ that is then fed into a last layer that corresponds to a classical learning task such as linear or logistic regression in order to predict the label $f(x^{(i)})$. One can also devise generalizations of (2.1), for instance in which matrix-vector multiplications are replaced by discrete convolutions in order to reflect other common architectures such as convolutional neural networks [GBC16, Chapter 9]. The key element that all these models share is that they all have *skip-connections*, namely, the previous step $x_i(k)$ appears explicitly in the iteration for the next one.

One upside of (2.1), which is the one of interest to our narrative, is that the layer index k can naturally be interpreted as a time variable, motivating the continuous-time analogue

$$\begin{cases} \dot{x}(t) = w(t)\sigma(a(t)x(t) + b(t)) & \text{for } t \in (0, T) \\ x(0) = x. \end{cases} \tag{2.2}$$

These are dubbed *neural ordinary differential equations* (neural ODEs). Since their introduction in [CRBD18, E17, HR17], neural ODEs have emerged as a flexible mathematical framework to implement and study ResNets. We use this abstraction here for convenience, as dynamical systems are more naturally defined in continuous time. Although this approach is helpful for exposition, we emphasize that all the results presented can also be derived in discrete time using the same techniques.

2.2. **The interacting particle system.** Unlike ResNets, which operate on a single input vector $x(0) \in \mathbb{R}^d$ at a time, Transformers operate on a sequence of vectors of length n, namely, $(x_i(0))_{i\in[n]} \in (\mathbb{R}^d)^n$. This perspective is rooted in natural language processing, where each vector represents a word, and the entire sequence a sentence or a paragraph. In particular, it allows to process words together with their context. A sequence element $x_i(0) \in \mathbb{R}^d$ is called a *token*, and the entire sequence $(x_i(0))_{i\in[n]}$ a *prompt*. We use the words "token" and "particle" interchangeably.

All practical implementations of Transformers make use of *layer normalization* [BKH16], most commonly in the form of *root mean square (RMS) normalization* [ZS19]. (See lines 105–116 in [Mis24] for instance.) RMS normalization takes the sequence of tokens output after each layer, divides each token by its Euclidean norm[1] (plus a small parameter to avoid a possible division by zero), and multiplies the result by a trained diagonal matrix. This process aims to ensure that tokens do not diverge, thus avoiding rounding errors and overflow. The result is an evolution on a time-varying axis-aligned ellipsoid. To simplify the presentation and obtain insight and precise results, we assume that the trained diagonal matrix is equal to the idenitity, so that we work on the unit sphere $\mathbb{S}^{d-1}$ throughout. This simplification is justified empirically in the trained ALBERT XLarge v2 model described in Figure 1, wherein this diagonal matrix is constant over all layers, with entries of mean value equal to 0.44 and standard deviation equal to 0.008. Furthermore, current text embeddings provided by OpenAI, namely `text-embedding-3-small` and `text-embedding-3-large`, return norm-one embedding vectors. While we can only speculate as to the actual implementation of these models, this is an indication that layer normalization could be as simple as the one used in our toy model.

A Transformer is then a flow map on $(\mathbb{S}^{d-1})^n$: the input sequence $(x_i(0))_{i\in[n]} \in (\mathbb{S}^{d-1})^n$ is an initial condition which is evolved through the dynamics

$$\dot{x}_i(t) = \mathbf{P}^{\perp}_{x_i(t)} \left(\frac{1}{Z_{\beta,i}(t)} \sum_{j=1}^{n} e^{\beta\langle Q(t)x_i(t), K(t)x_j(t)\rangle} V(t)x_j(t) \right) \tag{2.3}$$

for all $i \in [n]$ and $t \geqslant 0$. (We refer the reader to (2.8) and Section 2.3 for the full model.) Here and henceforth

$$\mathbf{P}^{\perp}_x y = y - \langle x, y\rangle x$$

[1]The original form instead consisted in an entry-wise standardization of every token, namely subtracting the mean of all tokens, then dividing by the standard deviation.

denotes the projection of $y \in \mathbb{R}^d$ onto $\mathrm{T}_x\mathbb{S}^{d-1}$. The *partition function* $Z_{\beta,i}(t) > 0$ reads

$$Z_{\beta,i}(t) = \sum_{k=1}^{n} e^{\beta\langle Q(t)x_i(t), K(t)x_k(t)\rangle}. \tag{2.4}$$

where $(Q(\cdot), K(\cdot), V(\cdot))$ (standing for Query, Key, and Value) are parameter matrices learned from data, and $\beta > 0$ a fixed number intrinsic to the model[2], which, can be seen as an inverse temperature using terminology from statistical physics. Note that $Q(\cdot), K(\cdot)$ need not be square.

The interacting particle system (2.3)–(2.4), a simplified version of which was first written down in [LLH+20, DGCC21, SABP22], importantly contains the true novelty that Transformers carry with regard to other models: the *self-attention mechanism*

$$A_{ij}(t) := \frac{e^{\beta\langle Q(t)x_i(t), K(t)x_j(t)\rangle}}{Z_{\beta,i}(t)}, \qquad (i,j) \in [n]^2, \tag{2.5}$$

which is the nonlinear coupling mechanism in the interacting particle system. The $n \times n$ stochastic matrix $A(t)$ (rows are probability vectors) is called the *self-attention matrix*. The wording *attention* stems from the fact that $A_{ij}(t)$ captures the attention given by particle i to particle j relatively to all particles $\ell \in [n]$. In particular, a particle pays attention to its neighbors where neighborhoods are dictated by the matrices $Q(t)$ and $K(t)$ in (2.5).

It has been observed numerically that the probability vectors $(A_{ij}(\cdot))_{j\in[n]}$ (for $i \in [n]$) in a trained self-attention matrix exhibit behavior related to the syntactic and semantic structure of sentences in natural language processing tasks (see [VSP+17, Figures 3-5]). To illustrate our conclusions as pedagogically as possible, throughout the paper we focus on a simplified scenario wherein the parameter matrices (Q, K, V) are constant, and even all equal to the identity unless stated otherwise, resulting in the dynamics

$$\dot{x}_i(t) = \mathbf{P}^{\perp}_{x_i(t)} \left(\frac{1}{Z_{\beta,i}(t)} \sum_{j=1}^{n} e^{\beta\langle x_i(t), x_j(t)\rangle} x_j(t) \right) \tag{SA}$$

for $i \in [n]$ and $t \geqslant 0$ and, as before

$$Z_{\beta,i}(t) = \sum_{k=1}^{n} e^{\beta\langle x_i(t), x_k(t)\rangle}. \tag{2.6}$$

The appearance of clusters in Transformers is actually corroborated by numerical experiments with trained models (see Figure 1). While we focus on a much simplified model, numerical evidence shows that the clustering phenomenon looks qualitatively the same in the cases $Q = K = V = \lambda I_d$, $\lambda > 0$, and generic random (Q, K, V) (see Figures 3 and 5 for instance). We refer the interested reader directly to Sections 4, 5, 6; here, we continue the presentation on the modeling of different mechanisms appearing in the Transformer architecture.

[2]In practical implementations the inner products are multiplied by $d^{-\frac{1}{2}}$, which along with the typical magnitude of Q, K leads to the appearance of β.

[3]ALBERT XLarge v2 contains all the mechanisms described in this text, namely, is a system of the form (2.8) (or rather the discretization thereof) with 12 or 24 layers. The sequence length n is of the order of 512 or 1024, and the tokens evolve in $\mathbb{R}^{4096}$. The dynamics are therefore high-dimensional, lending weight to assumptions made later on (Section 6).